# Taller — Pilas, colas y listas enlazadas

**SD1004 · Estructura de Datos** — Institución Universitaria Pascual Bravo

**Estudiante:** Mariana Berrio Macias

## Parte 1 — Reconocer la estructura adecuada

1. *Un editor de texto guarda las acciones recientes para permitir deshacer primero la última acción realizada.*

   **Estructura:** PILA

   La acción que se debe deshacer primero es la más reciente. La pila sigue la regla LIFO (último en entrar, primero en salir): cada acción se apila y pop() entrega exactamente la última registrada.

2. *Las personas esperan su turno en una fila de atención y se atiende primero a quien llegó primero.*

   **Estructura:** COLA

   Aquí aplica FIFO (primero en entrar, primero en salir). La persona nueva se ubica al final y se atiende por el frente, igual que en un sistema de turnos. Si se usara una pila, atenderían primero al último que llegó, lo cual sería injusto.

3. *Una aplicación recorre las canciones de una lista y, al terminar la última, vuelve automáticamente a la primera.*

   **Estructura:** LISTA CIRCULAR

   En la lista circular el sig del último nodo no apunta a nulo, sino al primer nodo. Por eso, al terminar la última canción, el paso "siguiente" lleva solo a la primera, sin agregar una condición como "si llegué al final, volver a la cabeza". El comportamiento queda en la forma de la estructura.

4. *Un sistema necesita recorrer una colección de registros en un único sentido y agregar elementos al comienzo con frecuencia.*

   **Estructura:** LISTA SIMPLE

   Cada nodo guarda ant y sig, así que desde cualquier posición se llega a los dos vecinos en un paso. En una lista simple, ir a la anterior obligaría a recorrer desde la cabeza hasta encontrar el nodo que apunta a la actual, que es O(n).

5. *Un reproductor permite pasar a la canción siguiente y a la anterior desde cualquier canción.*

   **Estructura:** LISTA DOBLE

   Cada nodo guarda ant y sig, así que desde cualquier posición se llega a los dos vecinos en un paso. En una lista simple, ir a la anterior obligaría a recorrer desde la cabeza hasta encontrar el nodo que apunta a la actual, que es O(n).

## Parte 2 — Pilas (LIFO)

### 2.1 Sigue las operaciones

Dibujo de la pila

| | **1. push(4)** | **2. push(9)** | **3. push(2)** | **4. pop()** | **5. push(7)** | **6. peek()** | **7. pop()** | **8. pop()** |
|---|---|---|---|---|---|---|---|---|
| **Nivel 3** | | | **2** ← tope | | **7** ← tope | **7** ← tope | | |
| **Nivel 2** | | **9** ← tope | 9 | **9** ← tope | 9 | 9 | **9** ← tope | |
| **Nivel 1 (base)** | **4** ← tope | 4 | 4 | 4 | 4 | 4 | 4 | **4** ← tope |

Tabla de seguimiento

| **Paso** | **Operación** | **Resultado** | **Contenido (base → tope)** | **Tope actual** |
|---|---|---|---|---|
| 1 | push(4) | No devuelve nada | [4] | 4 |
| 2 | push(9) | No devuelve nada | [4, 9] | 9 |
| 3 | push(2) | No devuelve nada | [4, 9, 2] | 2 |
| 4 | pop() | Devuelve **2** | [4, 9] | 9 |
| 5 | push(7) | No devuelve nada | [4, 9, 7] | 7 |
| 6 | peek() | Devuelve **7** (no lo retira) | [4, 9, 7] | 7 |
| 7 | pop() | Devuelve **7** | [4, 9] | 9 |
| 8 | pop() | Devuelve **9** | [4] | 4 |

**Estado final:** en la pila queda únicamente el elemento 4, que es también el tope, por ser el único.

### 2.2 Casos límite

1. **pop() sobre pila vacía:** no hay nada que retirar, lo que se conoce como underflow. Si no se controla, el programa intenta leer un tope inexistente y falla (o devuelve basura). Se evita validando estaVacia() antes de retirar. Si está vacía, se lanza un error o se devuelve nulo sin tocar la pila.
2. **peek() sobre pila vacía:** pasa lo mismo, no existe tope. Debe devolver nulo o un error, y no un número cualquiera, porque se confundiría con un dato real.
3. **Pseudocódigo:**

```
funcion deshacer(pila)
   si pila.estaVacia() entonces
      escribir "No hay acciones para deshacer"
      devolver nulo
   sino
      accion = pila.pop()
      devolver accion
   fin si
fin funcion
```

### 2.3 Aplicación: deshacer acciones

Las acciones se apilan en el orden en que se realizan: escribir, borrar, pegar.

| | **Antes de deshacer** | **Después del 1.er deshacer** | **Después del 2.º deshacer** |
|---|---|---|---|
| **Nivel 3** | **pegar** ← tope | | |
| **Nivel 2** | borrar | **borrar** ← tope | |
| **Nivel 1 (base)** | escribir | escribir | **escribir** ← tope |

- **1.er deshacer:** sale y se revierte **pegar**.
- **2.º deshacer:** sale y se revierte **borrar**.
- Queda solo **escribir**.

**¿Qué acción se deshace primero?** Pegar, porque fue la última acción realizada.

**¿Por qué una pila representa este comportamiento?** Las acciones entran en orden cronológico y solo se puede sacar la del tope, que siempre es la más reciente. Así salen en orden inverso al

que ocurrieron, que es lo que se espera de “deshacer”. Con una cola, el primer deshacer revertiría escribir (la más antigua) y el documento quedaría incoherente, porque se quitaría algo sobre lo que ya se hicieron otros cambios.

## Parte 3 — Colas (FIFO)

### 3.1 Simula la fila de atención

| Paso | Operación | Devuelve | Cola dibujada (frente → final) | Frente | Final |
|---|---|---|---|---|---|
| 1 | encolar(“Ana”) | – | [Ana] → nulo | Ana | Ana |
| 2 | encolar(“Luis”) | – | [Ana] → [Luis] → nulo | Ana | Luis |
| 3 | encolar(“Sol”) | – | [Ana] → [Luis] → [Sol] → nulo | Ana | Sol |
| 4 | desencolar() | **Ana** | [Luis] → [Sol] → nulo | Luis | Sol |
| 5 | encolar(“Mia”) | – | [Luis] → [Sol] → [Mia] → nulo | Luis | Mia |
| 6 | desencolar() | **Luis** | [Sol] → [Mia] → nulo | Sol | Mia |
| 7 | encolar(“Teo”) | – | [Sol] → [Mia] → [Teo] → nulo | Sol | Teo |

**Quedan esperando, en orden:** Sol (frente), Mia y Teo (final).

### 3,2 Operaciones y casos límite

1. Pseudocódigo

```
funcion atender(cola)
   si cola.estaVacia() entonces
      escribir "Cola vacía: no hay a quién atender"
      devolver nulo
   fin si
   persona = cola.frente.dato
   cola.frente = cola.frente.siguiente
   si cola.frente == nulo entonces
      cola.final = nulo
   fin si
   devolver persona
fin función
```

2. ¿Qué debería ocurrir si se intenta desencolar una cola vacía?
   Como no hay nadie en la fila, no hay a quién sacar. Si el programa lo intenta igual, frente es nulo y al leer frente.dato se produce un error (a esto le dicen underflow), y además la cola podría quedar en un estado raro. Yo lo manejaría así:

   1. Antes de sacar a alguien, reviso con estaVacia() si frente == nulo.
   2. Si está vacía, no toco nada: frente y final se quedan como están.
   3. Aviso que no hay nadie, ya sea devolviendo nulo o lanzando un error. Yo usaría el error cuando quien llama a la función nunca debería desencolar sin revisar antes, y nulo cuando es normal que a veces la fila esté vacía.

3.

Aquí hay que tener cuidado, porque si solo miramos el orden en que salen los datos, agregar por un lado y sacar por el lado contrario sí sigue siendo FIFO: el que entró primero sigue siendo el primero en salir. Lo único que cambia es cuál extremo le llamamos "frente". Pero con las definiciones del taller, donde `encolar` agrega al final y `desencolar` saca del frente, hacerlo al revés no representa bien la fila por estas razones:

1. **Contradice la definición:** agregar al frente sería ponerse justo donde atienden, o sea, colarse delante de todos. Eso no pasa en una fila real, donde el que llega se pone detrás.
2. **Es más costoso en una lista simple:** sacar del final obliga a recorrer toda la lista hasta el penúltimo nodo para hacerlo apuntar a `nulo`, y eso es O(n). Sacar del frente es O(1), porque solo se mueve `frente`.
3. **Lo que sí dañaría el FIFO** es agregar y sacar por el *mismo* lado, porque eso ya sería una pila (LIFO) y atenderían primero al último que llegó.

## Parte 4 — Listas enlazadas simples

### 4.1 Construye y modifica una lista

| Paso | Operación | Qué cambia | Lista dibujada |
|---|---|---|---|
| 1 | Insertar 18 al final | No hay nodos, así que el 18 es primero y último. `cabeza` lo apunta y su `sig` es `nulo`. | cabeza → [18 \| •] → nulo |
| 2 | Insertar 7 al comienzo | `7.sig = cabeza` (apunta al 18) y luego `cabeza = 7`. | cabeza → [7 \| •] → [18 \| •] → nulo |
| 3 | Insertar 24 al final | Se recorre hasta el último nodo (el 18), se hace `18.sig = 24` y el 24 apunta a `nulo`. | cabeza → [7 \| •] → [18 \| •] → [24 \| •] → nulo |
| 4 | Insertar 13 después del 7 | Primero `13.sig = 7.sig` (el 13 apunta al 18) y luego `7.sig = 13`. | cabeza → [7 \| •] → [13 \| •] → [18 \| •] → [24 \| •] → nulo |
| 5 | Eliminar el 18 | Su predecesor es el 13: `13.sig = 18.sig` (apunta al 24). El 18 queda desconectado. | cabeza → [7 \| •] → [13 \| •] → [24 \| •] → nulo |

**Diagrama final** (el nodo final apunta explícitamente a nulo):

**cabeza → [7 | •] → [13 | •] → [24 | •] → nulo**

### 4.2 Diseña operaciones

1. <u>Recorrer e imprimir</u>

   funcion recorrerEImprimir(cabeza)

```
    auxiliar = cabeza
    mientras auxiliar != nulo hacer
        escribir auxiliar.dato
        auxiliar = auxiliar.siguiente
    fin mientras
fin funcion
```

2. Buscar un valor

```
funcion buscar(cabeza, valorBuscado)
    auxiliar = cabeza
    mientras auxiliar != nulo hacer
        si auxiliar.dato == valorBuscado entonces
            devolver Verdadero
        fin si
        auxiliar = auxiliar.siguiente
    fin mientras
    devolver Falso
fin función
```

3. Insertar al comienzo

```
funcion insertarAlComienzo(cabeza, nuevoValor)
    nuevoNodo = crearNodo(nuevoValor)
    nuevoNodo.siguiente = cabeza
    cabeza = nuevoNodo
    devolver cabeza
fin funcion
```

### 4.3 Comprueba los borde

1. **Lista vacía.** cabeza es nulo, entonces nuevoNodo.siguiente = nulo y cabeza = nuevoNodo. El nodo queda como único elemento y apunta a nulo. El mismo código funciona sin caso especial.

| | **Diagrama** |
|---|---|
| Antes | cabeza → nulo |
| Después | cabeza → [5 \| •] → nulo |

2. **Lista con uno o más nodos.** `nuevoNodo.siguiente` copia la dirección del antiguo primer nodo y `cabeza` pasa a `nuevoNodo`. Los demás enlaces no se tocan

| | **Diagrama** |
|---|---|
| Antes | cabeza → [7 \| •] → [13 \| •] → nulo |
| Después (insertar 2) | cabeza → [2 \| •] → [7 \| •] → [13 \| •] → nulo |

3. **Eliminar el primer nodo.** Solo se actualiza una referencia: `cabeza = cabeza.siguiente`.

**Diagrama**

Antes cabeza → [7 | •] → [13 | •] → [24 | •] → nulo
Después cabeza → [13 | •] → [24 | •] → nulo (el nodo 7 queda desconectado)

Si la lista tenía un solo nodo, `cabeza.siguiente` es `nulo`, así que `cabeza` queda en `nulo` y la lista queda vacía, lo cual es correcto. El nodo desconectado lo libera el recolector de basura en lenguajes como Java o Python. En C o C++ hay que liberarlo manualmente (`free` o `delete`).

## Parte 5: Listas circulares

### 5.1 Reproduce un recorrido circular

1. **Lista circular inicial:**

   El sig de P3 apunta a P1, no a nulo.

   **[P1 | •] → [P2 | •] → [P3 | •] ↺ vuelve a P1**

2. **Registro de los primeros 7 turnos.**
   P2 termina en el *segundo turno que recibe*, no en el turno 2 global.

| Turno | Recibe | Qué ocurre | Siguiente |
|---|---|---|---|
| 1 | P1 | Sigue activo | P2 |
| 2 | P2 | Sigue activo (1.er turno de P2) | P3 |
| 3 | P3 | Sigue activo | P1 (por el enlace circular) |
| 4 | P1 | Sigue activo | P2 |
| 5 | P2 | **Termina** (2.º turno de P2) y se retira | P3 |
| 6 | P3 | Sigue activo | P1 |
| 7 | P1 | Sigue activo | P3 (P2 ya no está) |

3. **Dibuja la lista después de retirar P2. Verifica que el último nodo siga apuntando al primero:**

   **[P1 | •] → [P3 | •] ↺ vuelve a P1**

4. **¿Qué condición usarías para detener un recorrido de todos los nodos, dado que no se llega a nulo?**
   En una lista circular nunca se llega a nulo, entonces si uso mientras actual != nulo el ciclo no termina nunca. Lo que hago es guardar el nodo donde empecé y parar cuando vuelva a él, o sea, cuando actual sea igual a ese nodo de partida.

```
nodoInicio = cabeza
actual = nodoInicio
hacer
   procesar(actual)
   actual = actual.siguiente
mientras actual != nodoInicio
```

Otra forma es contar los nodos y repetir el ciclo esa cantidad de veces. Y antes de todo hay que revisar que la lista no esté vacía.

### 5.2 Casos especiales al eliminar

1. Lista vacía

   cabeza es nulo y no hay enlaces por actualizar. Se informa o se devuelve nulo. Se valida primero para no acceder a cabeza.siguiente sobre un nulo.

2. Un solo nodo

   El nodo se apunta a sí mismo (A.sig = A). Al eliminarlo no hay a quién enlazar, así que cabeza = nulo y la lista queda vacía.

3. Eliminar el nodo que se considera primero (dos o más nodos)

   Se busca el último nodo (ult, el que tiene sig = cabeza). Luego ult.sig = cabeza.sig y cabeza = cabeza.sig.

4. Eliminar el último nodo (dos o más nodos)

   Se itera hasta el penúltimo (pred) y se hace pred.sig = cabeza, cerrando el círculo.

**Diagrama del caso de un solo nodo:**

**Diagrama**

Antes [A | •] ↺ vuelve a A (A.sig = A)
Después cabeza → nulo (lista vacía)

**Diagrama del caso de dos o más nodos (eliminar el primero, A):**

**Diagrama**

Antes (cabeza = A) [A | •] → [B | •] → [C | •] ↺ vuelve a A
Después (cabeza = B) [B | •] → [C | •] ↺ vuelve a B (C.sig ahora es B)

Lo que cambia es el enlace del *último* nodo, no el del eliminado. Si se olvida, C seguiría apuntando a A, que ya no pertenece a la lista.

## Parte 6: Listas doblemente enlazadas

### 6.1 Recorrido en ambos sentidos

| Nodo | ant | dato | sig | Observación |
|---|---|---|---|---|
| 1 | nulo | **Lunes** | Martes | Es la `cabeza` |
| 2 | Lunes | **Martes** | Miércoles | |
| 3 | Martes | **Miércoles** | Jueves | |
| 4 | Miércoles | **Jueves** | nulo | Es la `cola` |

**Esquema:** nulo ⇄ Lunes ⇄ Martes ⇄ Miércoles ⇄ Jueves ⇄ nulo

- **Recorrido hacia adelante** (desde `cabeza`, siguiendo `sig` hasta nulo): Lunes → Martes → Miércoles → Jueves.
- **Recorrido hacia atrás** (desde `cola`, siguiendo `ant` hasta nulo): Jueves → Miércoles → Martes → Lunes.

### 6.2 Inserta y elimina

1. **Insertar Domingo antes de Lunes.** Domingo será el nuevo primer nodo. Se actualizan cuatro referencias:

- Domingo.ant = nulo
- Domingo.sig = Lunes
- Lunes.ant = Domingo (antes era nulo)
- cabeza = Domingo

Resultado: nulo ⇄ Domingo ⇄ Lunes ⇄ Martes ⇄ Miércoles ⇄ Jueves ⇄ nulo

2. **Insertar Viernes después de Jueves.** Viernes será el nuevo último nodo. Se actualizan:

- Jueves.sig = Viernes (antes era nulo)
- Viernes.ant = Jueves
- Viernes.sig = nulo
- cola = Viernes

Resultado: nulo ⇄ Domingo ⇄ Lunes ⇄ Martes ⇄ Miércoles ⇄ Jueves ⇄ Viernes ⇄ nulo

3. **Eliminar Miércoles.** Como Miércoles guarda a sus dos vecinos, no hay que recorrer nada. Se "puentea" el nodo:

- Martes.sig = Jueves (antes era Miércoles)

- Jueves.ant = Martes (antes era Miércoles)

**Diagrama**

| | |
|---|---|
| Antes | Martes ⇄ Miércoles ⇄ Jueves |
| Después | Martes ⇄ Jueves (Miércoles queda desconectado) |

**Resultado final después de las tres operaciones:**

| Nodo | ant | dato | sig |
|---|---|---|---|
| 1 | nulo | **Domingo** | Lunes |
| 2 | Domingo | **Lunes** | Martes |
| 3 | Lunes | **Martes** | Jueves |
| 4 | Martes | **Jueves** | Viernes |
| 5 | Jueves | **Viernes** | nulo |

### 6.3 Compara los enlaces

La ventaja de la lista doble es que, con el enlace ant, se puede recorrer hacia atrás (de Jueves a Lunes) sin volver a empezar desde la cabeza, algo que en una lista simple no se puede. También permite eliminar un nodo conocido, como Miércoles, sin buscar al que está antes. El costo es que cada nodo guarda una referencia extra, o sea, más memoria. Además hay más enlaces que mantener: al eliminar Miércoles tuve que corregir dos (Martes.sig y Jueves.ant) y no uno.

## Parte 7 — Análisis y elección

| Estructura y representación | Operación | Complejidad | Justificación breve |
|---|---|---|---|
| Pila enlazada con referencia al tope | Insertar en el tope | **O(1)** | Se crea el nodo, se enlaza al tope actual y se mueve `tope`. Son pasos fijos sin importar cuántos elementos haya. |
| Pila enlazada con referencia al tope | Retirar del tope | **O(1)** | `tope = tope.sig`. Se llega directo al tope por su referencia. |
| Cola enlazada con referencias al frente y al final | Encolar al final | **O(1)** | `final.sig = nuevo` y `final = nuevo`. La referencia al final evita recorrer. |
| Cola enlazada con referencias al frente y al final | Desencolar del frente | **O(1)** | `frente = frente.sig`. No depende de la longitud. |
| Lista simple sin referencia al último nodo | Insertar al comienzo | **O(1)** | Solo se modifica `cabeza` y el `sig` del nuevo nodo. |
| Lista simple sin referencia al último nodo | Insertar al final | **O(n)** | Sin referencia al último, hay que recorrer los n nodos para encontrarlo. |
| Lista doble con referencia a un nodo conocido | Desconectar ese nodo | **O(1)** | El nodo conoce a sus vecinos y basta enlazarlos entre sí. *Supuesto:* si está en un extremo, también se actualiza `cabeza` o `cola`. |

1. ¿Por qué una cola que solo guarda el frente tarda más en encolar?

Encolar es agregar después del último nodo, y si solo se conoce el frente, la única forma de llegar es empezar ahí y seguir los sig uno por uno hasta encontrar el nodo cuyo sig es nulo. Con n elementos son del orden de n pasos, O(n), y el costo crece con la fila. Si también se guarda final, se accede directo al último nodo y se agrega en pasos fijos, O(1).

2. ¿Qué ventaja ofrece la lista circular al volver repetidamente al primer elemento?

Como el último nodo apunta al primero, el paso "siguiente" ya lleva al inicio, y no hay que escribir la condición "si es el último, ir a la cabeza" ni mantener una referencia aparte para reiniciar. Eso evita errores por olvidar esa condición. Eso sí, para recorrer los nodos una sola vez sigue haciendo falta una condición de parada (volver al nodo inicial o contar los nodos). Es ideal para turnos, rotaciones o listas de reproducción en bucle.

3. ¿Qué información adicional almacena una lista doble y qué permite hacer?

Cada nodo guarda, además del dato y de sig, una referencia al nodo anterior. Con ella se puede:

1. Recorrer la lista en ambos sentidos, incluso desde el final.
2. Insertar o eliminar a partir de un nodo ya conocido sin buscar su predecesor (O(1)).
3. Pasar a la canción anterior directamente, como en el caso 5 de la Parte 1.

## Parte 8 — Reflexión

La forma en que se conectan los nodos define por dónde se puede entrar y salir de la estructura, y eso fija el orden en que se insertan y retiran los datos. En la pila todo ocurre por el tope, así que lo último en entrar es lo primero en salir, y por eso sirve para deshacer acciones o manejar el call stack. En la cola se usan dos extremos distintos, el final para ingresar y el frente para retirar, lo que produce el orden FIFO de una fila. La lista simple solo enlaza cada nodo con el siguiente, por lo que se avanza en un único sentido: insertar al comienzo es inmediato, pero llegar al final o retroceder obliga a recorrer. La lista doble agrega el enlace ant, que permite moverse en ambos sentidos y desconectar un nodo conocido en tiempo constante, a cambio de más memoria y más punteros que mantener. La lista circular cierra el último nodo sobre el primero, lo que la hace útil para recorridos repetitivos como el Round Robin de los sistemas operativos. Al final no hay una estructura mejor que las demás: cada una está optimizada para un patrón de acceso distinto